CloudFlow: Identifying Security-sensitive Data Flows in Serverless Applications

Giuseppe Raffa[(a)], Jorge Blasco[(b)], Dan O'Keeffe[(a)], Santanu Kumar Dash[(c)]
[(a)]Royal Holloway University of London, [(b)]Universidad Politécnica de Madrid, [(c)]University of Surrey

Serverless Computing Model

- Cloud providers manage the **infrastructure**, whilst developers are **responsible** for the application security.

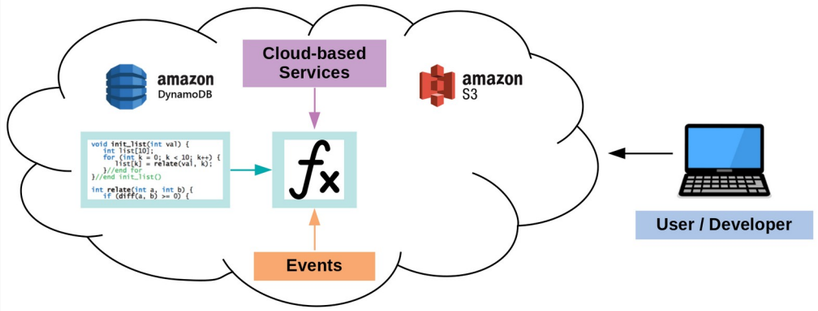

Fig. 1 – Serverless computing model.

Motivation

- Static analysis allows inspecting an application **without** executing it. Therefore, it can **prevent** the deployment of vulnerable code.

Challenges

- Serverless applications are **event-driven** and **interact** with the cloud platform. The application in Fig. 2, for instance, includes an event-triggered data flow that runs a system command with an **user-controlled** input.
- **Multiple** and **complex** permission mechanisms.
- The platform services' code is **proprietary**.

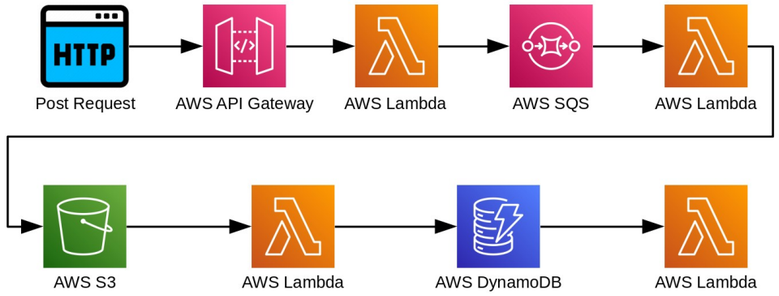

Fig. 2 – Example of vulnerable serverless application.

Our Work

- **CloudFlow** is a framework to statically detect **security-sensitive** data flows in serverless applications.

CloudFlow Analysis Pipeline

- Our framework extracts **permissions**, **events** and **handlers** (PEH) from the infrastructure code (Listing 1).
- CloudFlow subsequently identifies the **cloud API calls** in the application code. These are processed with **models** to extract their relevant inputs.
- By combining the obtained information, we **instrument** the application code with calls to event-triggered **handlers** (Listing 2).
- The instrumented code features a **synchronous-like** program execution flow, and can be analysed with a **general-purpose** tool.

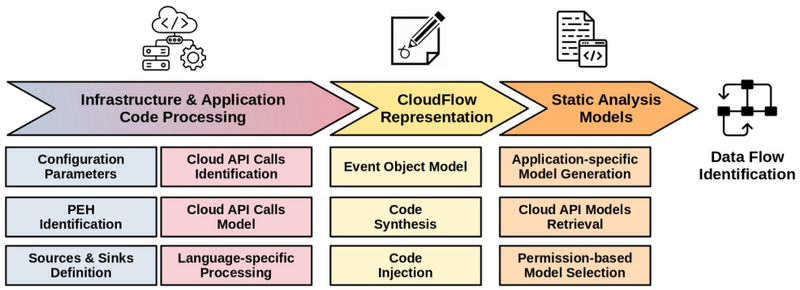

Fig. 3 – CloudFlow static analysis pipeline.

```
iamRoleStatements:
    - Effect: "Allow"
      Action:
        - "s3: ListBucket"
        - "s3: GetObject"
        - "s3: PutObject"
```

Listing 1 – Example of infrastructure code.

```
def onSQSMessage(event, context):
    # Application code
    s3_client.upload_file(localFile, …)          # Cloud API
    event = {'Records': [{'eventName': …}]}      # Initialization
    s3uploadhandler.onS3Upload(event, context)   # Handler call
```

Listing 2 – Example of instrumented cloud API.

Microbenchmark-based Evaluation

- We evaluate CloudFlow with **CloudBench**, a new suite of microbenchmarks (Table 1).
- CloudFlow passes **37 out of 40** microbenchmarks.
- The analysis of the failed microbenchmarks shows one **false positive** and two **false negatives**.

Category	Amount
Inter-procedural	30
Intra-procedural	9
Simple Applications	1

Table 1 – CloudBench microbenchmark suite.

Real-world Application Analysis

- We also analyse **104 real-world applications**.
- CloudFlow detects a total of **205** security-sensitive data flows, and **11** of them are **vulnerabilities** (Table 2).

Vulnerability Type	Total No.
Code Injection via CLI	2
Code Injection via HTTP	5
Exception Traceback Leakage	4

Table 2 – Summary of identified vulnerabilities.

Future Work

- We plan to increase the number of supported **cloud services** and **APIs**.
- To automatically create **API call models**, we intend to leverage an API documentation processing framework.
- To support different languages, we are evaluating alternative **underlying static analysis tools**.

giuseppe.raffa.2018@live.rhul.ac.uk